

# Riyadh Doha Poultry Analytics

Poultry-Intelligence-System-PLS-

FINAL PROJECT DOCUMENTATION · 2026

## From Raw Data to Predictive Intelligence

An end-to-end analytics and machine learning project for a real working poultry farm

---

**Tech Stack:** Excel · SQL · Python · Tableau · Power BI · Streamlit . landpage . ml



# Project Overview

## The Farm

Built for **Riyadh Doha**, a commercial layer poultry farm operating with approximately **75,000 birds** at scale.

## The Data

Transforms **5,200+ daily operational records** spanning **2019 to 2026** into a governed analytics platform.

## The Deliverables

Delivers dashboards, a trained ML model, a web app, and a hosted website — end to end.

## The Blueprint

Acts as a **complete blueprint** for small to mid-sized agricultural analytics operations.

# Headline Results

Seven years of farm operations distilled into four defining numbers — each a testament to disciplined data collection and rigorous analysis.

## 124M

**Total Eggs Produced**

Cumulative egg output across all flock cycles from 2019 to 2026.

## 316M EGP

**Total Revenue Generated**

Gross revenue across all egg sizes and sales channels over the full period.

## 312M EGP

**Cumulative Net Profit**

Net profit after all operational costs, feed, and overhead deductions.

## 0.82

**Model R<sup>2</sup> Score**

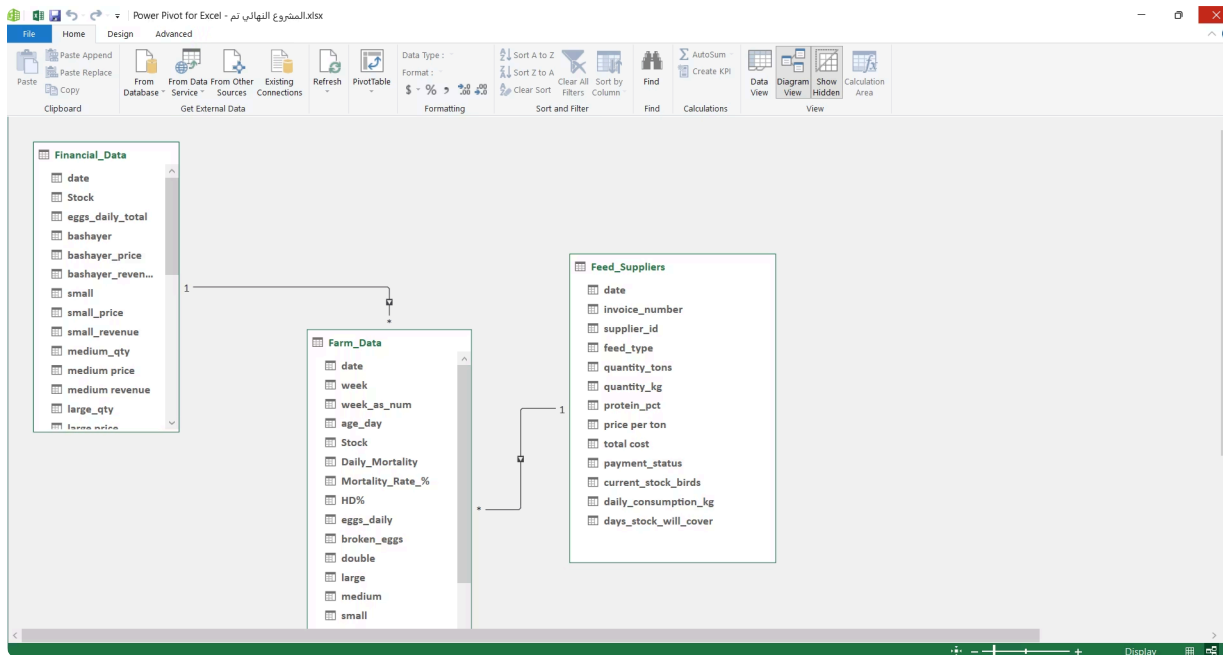
XGBoost model accuracy on unseen test weeks — strong predictive performance.

# System Architecture

A six-stage pipeline transforms a raw Excel workbook into live predictions — every layer purpose-built for reliability and scale.



Each stage is independently testable and replaceable — a modular architecture that scales from a single farm to an entire agricultural enterprise.



STAGE 1 · EXCEL

# Data Modeling in Excel

The source workbook contains three operational sheets that form the foundation of the entire analytics pipeline.

## → Three Source Sheets

Farm Data, Financial Data, and Feed Suppliers — each capturing a distinct operational domain.

## → Date Standardization

All date columns standardized to a single ISO-friendly format across every sheet.

## → Structural Inspection

Inspected merged cells, stray totals rows, and inconsistent date formats before any transformation.

## → Validation Pivots

Built pivot tables to confirm row counts per year and verified per-size egg quantities sum to the daily total.

# Cleaning Inside Excel

## Remove Text Placeholders

Removed text placeholders like N/A and dashes so Python reads them correctly as NaN during ingestion.

## Normalized Header Row

Created a clean header row stripped of trailing spaces and special characters for consistent downstream parsing.

## Power Query Join Prototype

Prototyped the Farm × Financial Data join inside Power Query first to validate the merge logic before Python.

## Feed Supplier Aggregation

Aggregated Feed Suppliers from per-invoice to per-day granularity to align with the farm's daily operational grain.

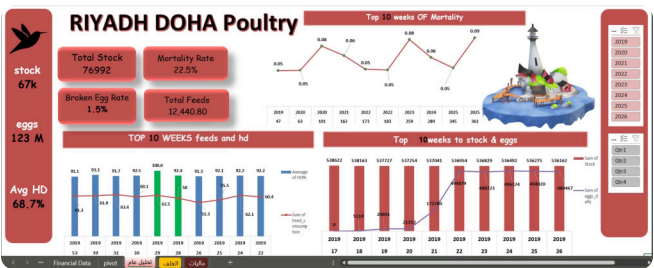
## Integrity Flagging

Highlighted rows that failed integrity checks in red for operator review — a human-readable audit trail.

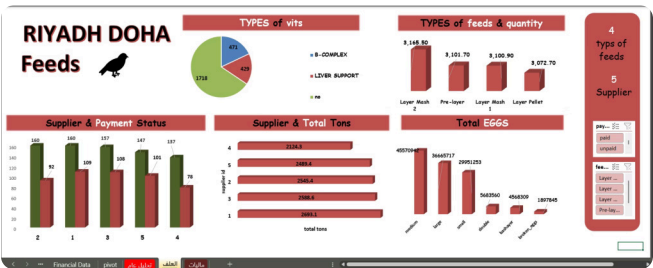
## Canonical Raw Input

Result: a clean, validated raw input file ready for the Python pipeline — the single source of truth.

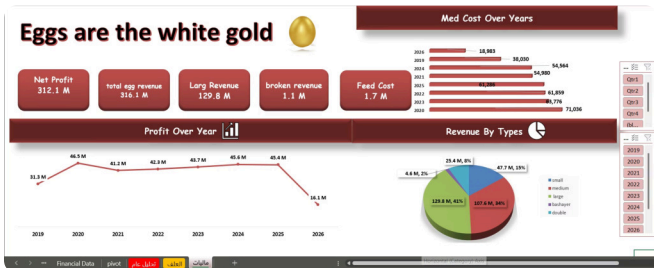
# These cards are for data analysis through Excel, 3 integrated dashboards



public analysis



feeds and suppliers

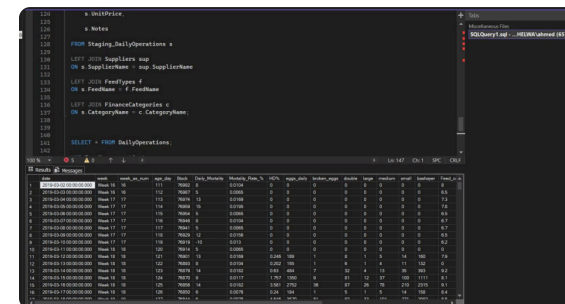
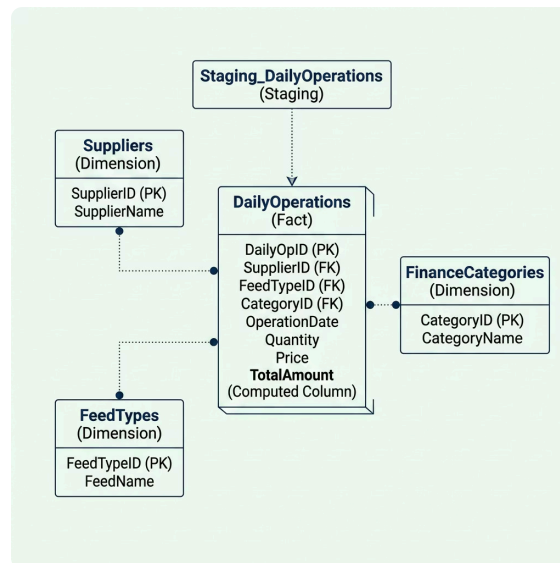


financial dash

# Database Design

A normalized SQL Server schema named `FarmManagementSystem` — engineered for integrity, speed, and extensibility.

**TotalAmount** stored as a **PERSISTED computed column** for fast aggregation without runtime recalculation.



## 3NF Schema

Third Normal Form with surrogate `IDENTITY` keys on every dimension table.



## Dimension Tables

Three dimensions: Suppliers, FeedTypes, and FinanceCategories.



## Staging Table

`Staging_DailyOperations` receives the flat file from Python as the entry point.



## Fact Table

Central `DailyOperations` fact table with foreign keys to every dimension.



# ETL and Population

A disciplined ETL sequence ensures referential integrity is established before the fact table is ever touched.

01

## Populate Suppliers

INSERT INTO Suppliers using SELECT DISTINCT SupplierName FROM staging — deduplication at source.

03

## Populate FinanceCategories

INSERT INTO FinanceCategories using SELECT DISTINCT CategoryName FROM staging.

05

## Row Count Verification

Final SELECT confirms row counts match the source workbook — zero tolerance for silent data loss.

02

## Populate FeedTypes

INSERT INTO FeedTypes using SELECT DISTINCT FeedName FROM staging — clean dimension before fact load.

04

## Load Fact Table

Fact table populated by joining staging back to all populated dimensions — foreign keys resolved at insert time.

06

## Schema Extensibility

Architecture supports adding new suppliers or feed products without any schema change — future-proof by design.

# Cleaning Pipeline

The `farm_project.py` script is the backbone of the data layer — transforming three messy sheets into one authoritative CSV.

1

## Sheet Ingestion

Loads all three sheets using `pandas.read_excel` into separate dataframes.

2

## Header Cleaning

`clean_header` function lowercases, trims, and removes parentheses from all column names.

3

## Date Coercion

Date columns coerced with `pd.to_datetime` across all three frames uniformly.

4

## Dedup and Fill

`df_finance` deduplicated; `df_farm` forward-filled to carry last valid sensor readings.

5

## Supplier Grouping

Suppliers grouped to daily totals before joining to align granularity across all frames.

6

## Three-Way Merge

Final three-way merge produces a single cleaned dataframe exported to the master CSV.

# Validation and **Weekly Aggregation**

## Validation Checks

### Range Plausibility

Logical check confirms profit and production ranges are plausible — catches outliers before modeling.

### Zero Duplicates

Duplicate row count verified to be exactly zero after the three-way merge completes.

### Null Inspection

Missing values inspected column by column using `isnull().sum()` — no silent gaps allowed.

## Weekly Aggregation

### ISO Week Grouping

Daily records grouped to ISO weeks using `dt.to_period('W')` for consistent temporal alignment.

### Summed Metrics

`net_profit` and `eggs` summed per week — additive measures aggregated correctly.

### Averaged Metrics

HD%, mortality, and stock averaged per week — rate measures handled with the correct aggregation function.

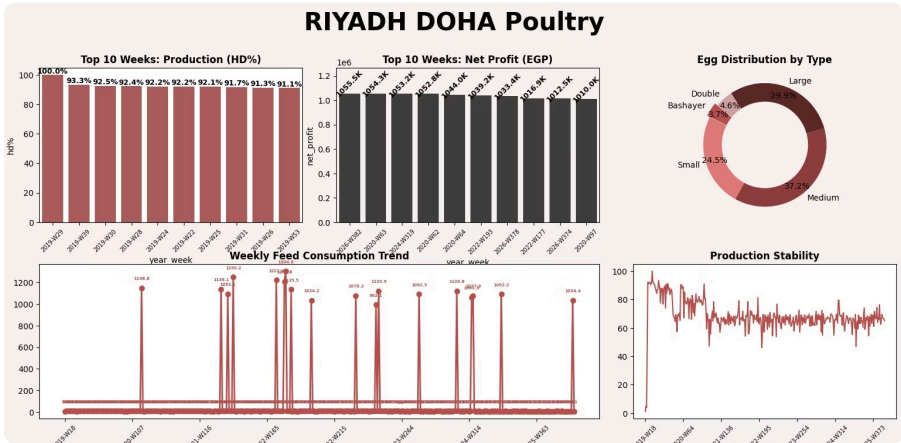


The final weekly dataframe becomes the unified input for both **visualization** and **ML modeling** — one source, two destinations.

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import matplotlib.ticker as ticker
6
7 df_finance = pd.read_excel("C:\Users\Haga Tech\Documents\poultry_farm_v2.xlsx", sheet_name="Financial Data")
8 df_farm = pd.read_excel("C:\Users\Haga Tech\Documents\poultry_farm_v2.xlsx", sheet_name="Farm Data")
9 df_suppliers = pd.read_excel("C:\Users\Haga Tech\Documents\poultry_farm_v2.xlsx", sheet_name="Feed Suppliers")
10
11 def clean_header(df):
12     df.columns = [str(col).strip().lower().replace(" ", "_").replace("(", "").replace(")", "").replace("'", "") for col in df.columns]
13     return df
14
15 df_finance = clean_header(df_finance)
16 df_finance['date'] = pd.to_datetime(df_finance['date'])
17 df_finance = df_finance.drop_duplicates()
18
19 df_farm = clean_header(df_farm)
20 df_farm['date'] = pd.to_datetime(df_farm['date'])
21 df_farm = df_farm.fillna(0)
22
23 df_suppliers = clean_header(df_suppliers)
24 df_suppliers['date'] = pd.to_datetime(df_suppliers['date'])
25
26 # Print data info
27 print(df_finance.info())
28 print(df_farm.info())
29 print(df_suppliers.info())
30
31 # Merge data
32 df_merged = df_finance.merge(df_farm, on='date', how='inner', suffixes=('', '_farm'))
33
34 df_final = pd.merge(df_merged, df_suppliers, on='date', how='left')
35
36 df_final['quantity_ton'] = df_final['quantity_ton'].fillna(0)
37 df_final['total_cost_egg'] = df_final['total_cost_egg'].fillna(0)
38
39 df_final.to_csv('poultry_cleaned_full.csv', index=False)
```

```
1 # Import libraries
2 import pandas as pd
3 import numpy as np
4
5 # Read data
6 df_finance = pd.read_excel("C:\Users\Haga Tech\Documents\poultry_farm_v2.xlsx", sheet_name="Financial Data")
7 df_farm = pd.read_excel("C:\Users\Haga Tech\Documents\poultry_farm_v2.xlsx", sheet_name="Farm Data")
8 df_suppliers = pd.read_excel("C:\Users\Haga Tech\Documents\poultry_farm_v2.xlsx", sheet_name="Feed Suppliers")
9
10 # Clean headers
11 def clean_header(df):
12     df.columns = [str(col).strip().lower().replace(" ", "_").replace("(", "").replace(")", "").replace("'", "") for col in df.columns]
13     return df
14
15 df_finance = clean_header(df_finance)
16 df_finance['date'] = pd.to_datetime(df_finance['date'])
17 df_finance = df_finance.drop_duplicates()
18
19 df_farm = clean_header(df_farm)
20 df_farm['date'] = pd.to_datetime(df_farm['date'])
21 df_farm = df_farm.fillna(0)
22
23 df_suppliers = clean_header(df_suppliers)
24 df_suppliers['date'] = pd.to_datetime(df_suppliers['date'])
25
26 # Print data info
27 print(df_finance.info())
28 print(df_farm.info())
29 print(df_suppliers.info())
30
31 # Merge data
32 df_merged = df_finance.merge(df_farm, on='date', how='inner', suffixes=('', '_farm'))
33
34 df_final = pd.merge(df_merged, df_suppliers, on='date', how='left')
35
36 df_final['quantity_ton'] = df_final['quantity_ton'].fillna(0)
37 df_final['total_cost_egg'] = df_final['total_cost_egg'].fillna(0)
38
39 df_final.to_csv('poultry_cleaned_full.csv', index=False)
```

```
1 # Import libraries
2 import pandas as pd
3 import numpy as np
4
5 # Read data
6 df_finance = pd.read_excel("C:\Users\Haga Tech\Documents\poultry_farm_v2.xlsx", sheet_name="Financial Data")
7 df_farm = pd.read_excel("C:\Users\Haga Tech\Documents\poultry_farm_v2.xlsx", sheet_name="Farm Data")
8 df_suppliers = pd.read_excel("C:\Users\Haga Tech\Documents\poultry_farm_v2.xlsx", sheet_name="Feed Suppliers")
9
10 # Clean headers
11 def clean_header(df):
12     df.columns = [str(col).strip().lower().replace(" ", "_").replace("(", "").replace(")", "").replace("'", "") for col in df.columns]
13     return df
14
15 df_finance = clean_header(df_finance)
16 df_finance['date'] = pd.to_datetime(df_finance['date'])
17 df_finance = df_finance.drop_duplicates()
18
19 df_farm = clean_header(df_farm)
20 df_farm['date'] = pd.to_datetime(df_farm['date'])
21 df_farm = df_farm.fillna(0)
22
23 df_suppliers = clean_header(df_suppliers)
24 df_suppliers['date'] = pd.to_datetime(df_suppliers['date'])
25
26 # Print data info
27 print(df_finance.info())
28 print(df_farm.info())
29 print(df_suppliers.info())
30
31 # Merge data
32 df_merged = df_finance.merge(df_farm, on='date', how='inner', suffixes=('', '_farm'))
33
34 df_final = pd.merge(df_merged, df_suppliers, on='date', how='left')
35
36 df_final['quantity_ton'] = df_final['quantity_ton'].fillna(0)
37 df_final['total_cost_egg'] = df_final['total_cost_egg'].fillna(0)
38
39 df_final.to_csv('poultry_cleaned_full.csv', index=False)
```



5 charts

# Stage 3: Python — Five-Panel Dashboard

PART 2 OF 2 · SLIDE 11

The Python dashboard consolidates five analytical panels into a single executive view — from production rankings and profit weeks to egg distribution, feed trends, and long-term HD stability — all rendered in the custom red and charcoal palette.



## Top 10 Production Weeks

Ranked by HD percentage — highest-performing flock weeks surfaced instantly



## Top 10 Profit Weeks

Ranked by net\_profit in thousands of EGP — financial peaks identified at a glance



## Egg Distribution Donut

Bashayer, Small, Medium, Large, and Double split visualized proportionally



## Feed Consumption Trend

Weekly feed line with per-week annotations tracking consumption patterns



## Production Stability

Long-term HD percentage drift monitored for flock health consistency

# Stage 4: Tableau — Public Dashboard

PART 2 OF 2 · SLIDE 12

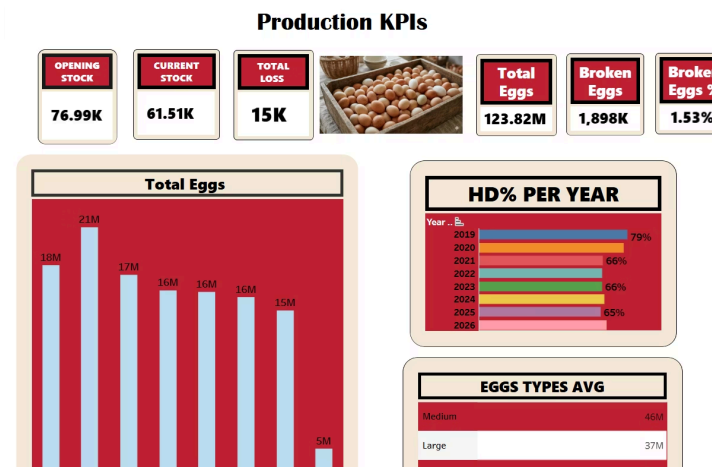
## Why Tableau Public?

Chosen for **free public hosting** and frictionless executive sharing — no login, no plugin, no friction. The live dashboard spans **2019 to 2026** with a drillable date filter accessible to any stakeholder.

 Live URL:  
[public.tableau.com/app/profile/asmaa.elshazly/viz/PRODUCTI/ONKPIS](https://public.tableau.com/app/profile/asmaa.elshazly/viz/PRODUCTI/ONKPIS)

## Interactive Features

- Daily HD percentage over time with a moving average overlay
- Egg-size mix shown as an interactive stacked bar by year
- Mortality vs Stock dual-axis line for flock stability
- Top-N week ranker with a parameter control so reviewers can change N
- Drillable date filter spanning 2019 to 2026



# Stage 5: Power BI — **Five** Report Pages

PART 2 OF 2 · SLIDE 13

Each of the five Power BI report pages is reachable through a **persistent navigation panel** that lives on every screen — ensuring seamless movement between executive summaries and granular analytics.

1

## Executive Overview

Surfaces headline KPIs, mortality rate, and feed vs water consumption at a glance

2

## Eggs Production

Decomposes size mix across all egg categories and tracks the broken-egg trend over time

3

## Feed Optimization

Compares received vs consumed feed volumes and monitors water consumption patterns

4

## Suppliers Analytics

Tracks invoices, paid vs unpaid balances, and individual supplier performance scores

5

## Financial Analytics

Reports revenue, cost gap, per-egg margin, and net profit across the full date range

# Stage 5: Power BI — Key Findings

PART 2 OF 2 · SLIDE 14

## Production Insights

**~70%**

Medium and large eggs together represent approximately 70% of total production

**41.19%**

Large eggs alone contribute 41.19% of total egg revenue — the dominant size tier

**Declining**

Broken-egg counts decline year on year, signaling improving quality control

## Financial & Supplier Insights

**39%**

Of supplier invoices remain unpaid and were flagged to finance for follow-up

**2022 Peak**

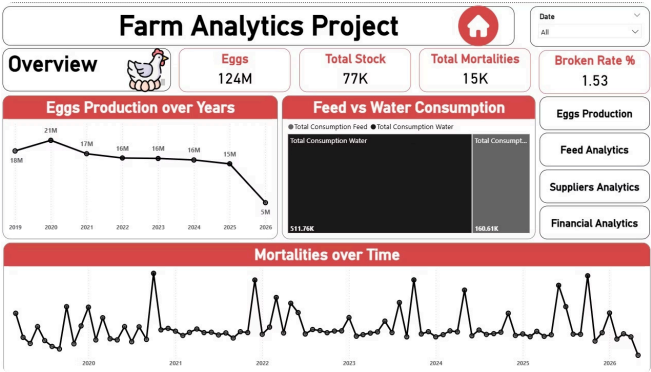
Feed consumption peaks in 2022 then declines in line with stock reductions

**41M–47M EGP**

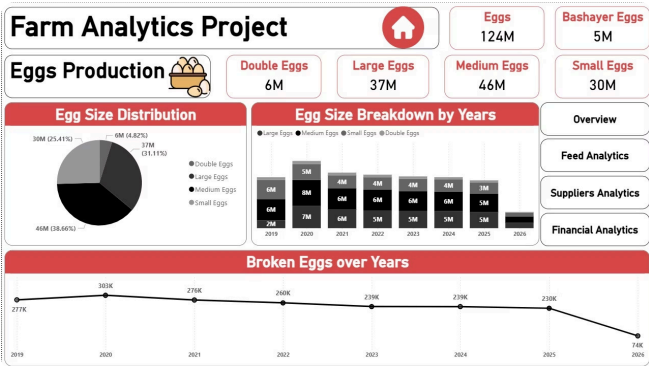
Net profit stays steady between 41M and 47M EGP from 2019 through 2025



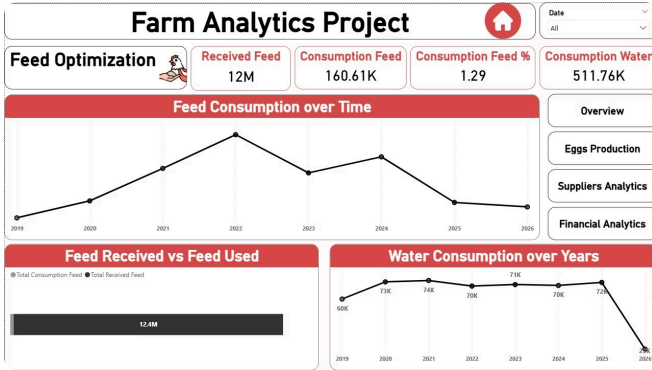
# 3 examples of dashboards on Power BI



OVERVIEW



EGGS PRODUCTION

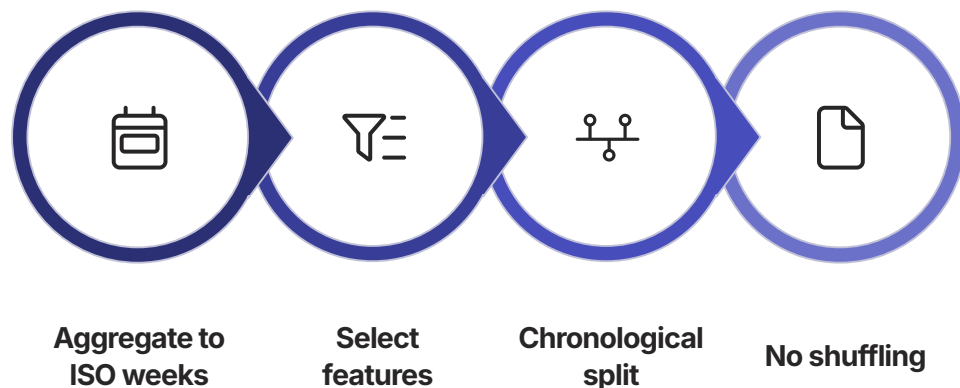


FEED OPTIMIZATION

## Stage 6: Machine Learning — Problem Framing

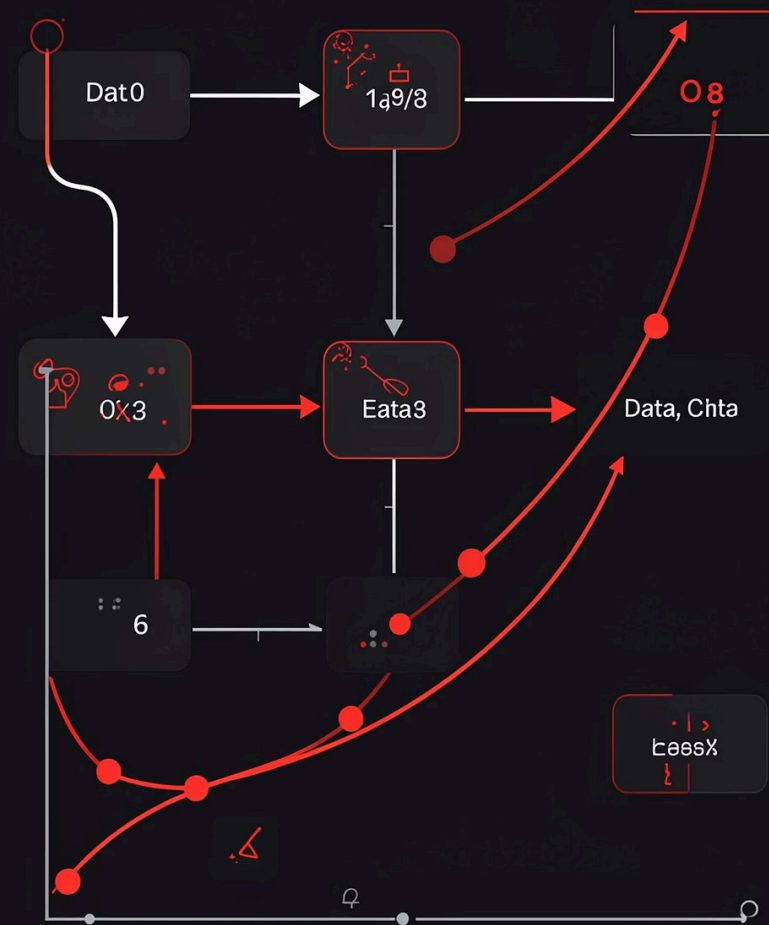
PART 2 OF 2 · SLIDE 15

A **supervised regression** approach was selected with **net\_profit** as the weekly prediction target. The methodology preserves the integrity of the time-series structure throughout every stage of preparation.



By aggregating daily rows to ISO weeks and splitting chronologically — with the final 20% held out as the test set and no shuffling applied — the model is evaluated on genuinely unseen future data, making performance metrics trustworthy for operational decision-making.

# Machine Learning Model Training



# Stage 6: **ML** — Feature Engineering & Model

PART 2 OF 2 · SLIDE 16

## Engineered Features

### → **age\_week**

Derived from  $\text{age\_day} \div 7$  — normalizes flock age into weekly units

### → **Lag Features**

profit\_lag and hd\_lag at horizons of 1, 2, and 4 weeks — capturing recent momentum

### → **Rolling Means**

Profit and HD percentage averaged over 2, 4, and 6-week windows for smoothed signals

## XGBoost Hyperparameters

### **250 Estimators**

Learning rate: 0.03 · Max depth: 3

### **Strong Regularization**

reg\_alpha: 4.0 · reg\_lambda: 8.0

### **Variance Reduction**

subsample: 0.70 · colsample\_bytree: 0.70

❏ XGBoost chosen for its strength in tabular regression on small, structured datasets with strong regularization to prevent overfitting.

# Stage 6: ML — Results & Feature Importance

PART 2 OF 2 · SLIDE 17

0.953

Train  $R^2$

Excellent in-sample fit across the training period

0.816

Test  $R^2$

Strong out-of-sample performance on held-out weeks

11.5K

Train MAE

11,564 EGP average error on training set

24.7K

Test MAE

24,666 EGP average error on unseen test weeks

## Feature Importance Findings

### 1 profit\_lag\_1 Dominates

Last week's profit is the single strongest predictive signal in the model

### 2 Rolling Profit Averages

2-week and 4-week rolling profit means capture short-term financial momentum effectively

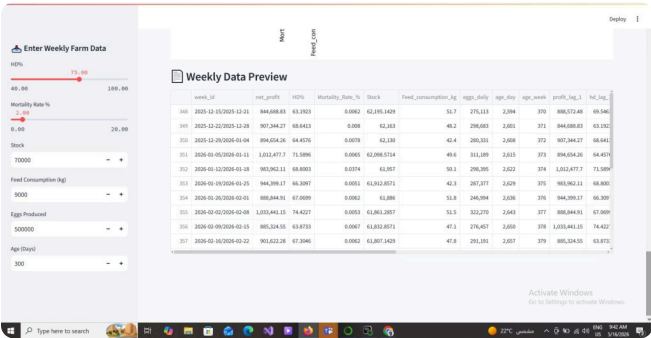
### 3 HD Percentage in Top Three

Validates flock productivity as a core driver of weekly net profit outcomes

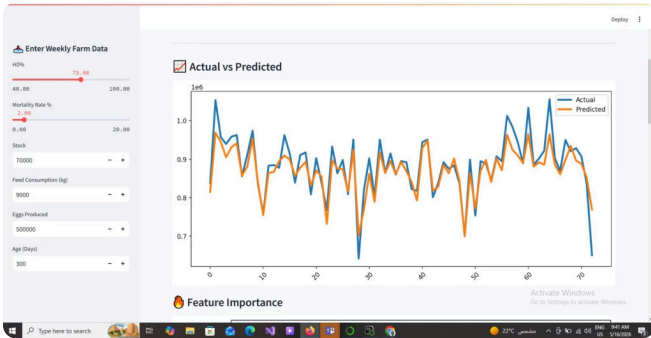
### 4 Secondary Corrections

Feed consumption and stock levels provide additional corrective signals; model serialized with joblib for deployment

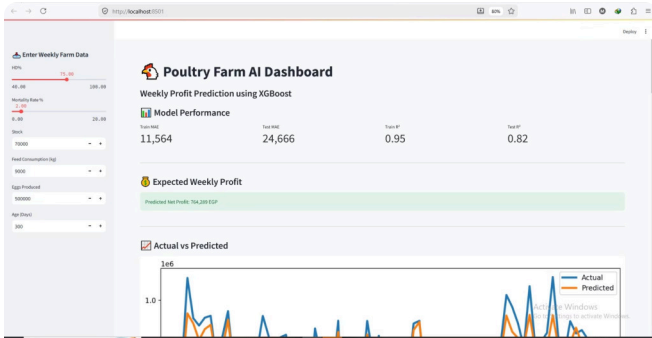
# 3 cards smart



weekly data preview



Actual predicted

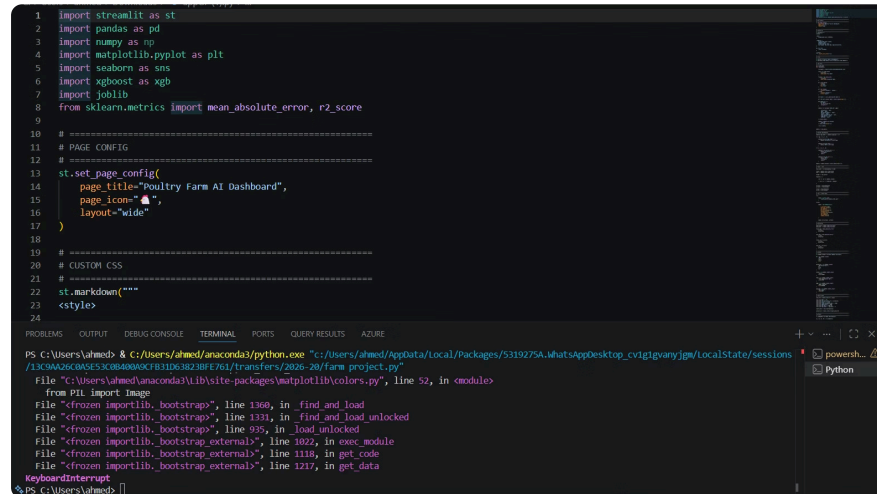


AI dashboard

# Stage 7: Streamlit — Predictive Web App

PART 2 OF 2 · SLIDE 18

The **appGP.py** Streamlit application brings the trained XGBoost model to life as an interactive, real-time prediction tool — with custom CSS maintaining full visual consistency with the dashboard palette.



```
1 import streamlit as st
2 import pandas as pd
3 import numpy as np
4 import matplotlib.pyplot as plt
5 import seaborn as sns
6 import xgboost as xgb
7 import joblib
8 from sklearn.metrics import mean_absolute_error, r2_score
9
10 # =====
11 # PAGE CONFIG
12 # =====
13 st.set_page_config(
14     page_title="Poultry Farm AI Dashboard",
15     page_icon="🐔",
16     layout="wide"
17 )
18 # =====
19 # CUSTOM CSS
20 # =====
21 st.markdown("""
22 <style>
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

## Design Principles

### No Submit Button

Real-time prediction updates on every input change — zero friction for the end user

### Full Transparency

Feature importance chart and correlation heatmap expose model logic to non-technical reviewers

### Consistent Palette

Custom CSS keeps the warm red and charcoal look identical to the BI dashboards and website

### Live Performance Cards

Train and Test MAE and  $R^2$  displayed prominently so users understand model confidence at all times

# Stage 8: **Live** Web Dashboard

PART 2 OF 2 · SLIDE 19

✓ Live URL: **dohaafarm.netlify.app** — No login. No plugin. No friction.

## Five Themed Navigation Tiles



**Executive Overview**



**Production & Egg Quality**



**Feed & Resource  
Optimization**



**Suppliers Analytics**



**Financial Analytics**

## Design Rationale

- Landing page mirrors the Power BI navigation with five themed tiles for instant familiarity
- Warm earth-tone palette communicates farm context rather than a trading floor aesthetic
- Mobile responsive so farm staff can read reports on phones directly in the field
- Frictionless front-door with no login or plug-in required for any stakeholder



## تحليلات مزرعة الدواجن

نظام المتابعة الذكي للمزرعة

بيانات حية

2019 - 2026 · 5,236 سجل

نظرة عامة

الإنتاج

صحة القطيع

العلف والموردون

الماليات

## النظرة العامة

أهم مؤشرات الأداء للمزرعة - مارس 2019 - مايو 2026

أبرز الإنجازات: بلغت المزرعة أعلى معدل إنتاج في مايو 2019 بنسبة وضع 93.2% وأكثر من 4.4 مليون بيضة في شهر واحد. الإنتاج التراكمي الكلي 247.6 مليون بيضة بصافي ربح 624.3 مليون جنيه مصري.



24.9M kg

إجمالي العلف المستلم



29,752

إجمالي النفوق



76,992

حجم القطيع الابتدائي



69.0%

متوسط نسبة الوضع HD%



624.3M

صافي الربح (جنيه)



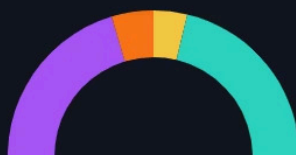
247.6M

إجمالي البيض المنتج

كل الأوقات

### توزيع أحجام البيض

الإنتاج التراكمي حسب الدرجة



7 سنوات

### الاتجاه الشهري لإنتاج البيض

إجمالي البيض المحقق شهرياً (2019-2026)





# Conclusion & Project Links

PART 2 OF 2 · SLIDE 20

One integrated project delivered through **three front-doors** — Power BI, Streamlit, and a live website — with a cleaned dataset, normalized database, trained model, and all dashboards fully reproducible.

## Key Achievements

### Lessons Learned

- Start in the user's native tool to maximize adoption
- Separate concerns — data, model, and presentation layers stay independent
- Keep palette consistent across every surface for brand coherence

## Future Work

### → Live Data Feed

Automate daily ingestion to replace manual CSV uploads with real-time pipeline

### → Weather Features

Integrate temperature and humidity data to improve model accuracy during seasonal shifts

### → Role-Based Auth

Add authentication layer so farm staff, finance, and executives see tailored views



**Website:** [dohaafarm.netlify.app](https://dohaafarm.netlify.app)

**Tableau:** [public.tableau.com/app/profile/asmaa.elshazly/viz/PRODUCTIONKPIS](https://public.tableau.com/app/profile/asmaa.elshazly/viz/PRODUCTIONKPIS)

### 0.82 Test R<sup>2</sup>

Decision-support grade accuracy for weekly budgeting and operational planning

### Three Front-Doors

BI dashboards, Streamlit app, and live website serve different stakeholder needs simultaneously

### Fully Reproducible

Cleaned dataset, normalized DB, trained model, and all dashboards are version-controlled and reproducible

رواد مصر الرقمية



**thank you**

team 1